

Contents

1	Conversation Log — 2026-05-31 09:54	1
1.1	User Requests This Session	1
1.1.1	1. Diagnose <code>lightgbm</code> build failure when running <code>bootstrap.py</code>	1
1.1.2	2. Understand <code>venv</code> / Docker relationship	1
1.1.3	3. Analyse <code>builds_bootstrap</code> repo for dockerization	2
1.1.4	4. Decision: remove Claude dependency entirely	2
1.1.5	5. Add LightGBM, XGBoost, CatBoost as standard libraries	2
1.1.6	6. Implement all changes in <code>builds_bootstrap</code> repo	2
1.1.7	7. Preserve Claude-based version as git tag	3
1.1.8	8. Explain <code>./run.sh</code> vs <code>./start.sh</code>	3
1.1.9	9. Update all documentation	3
1.1.10	10. Add friendly Docker-not-running error to <code>run.sh</code>	3
1.2	Final Architecture After This Session	4
1.3	All Commits This Session	4

1 Conversation Log — 2026-05-31 09:54

Session focused on `builds_bootstrap` repo — diagnosing a pipeline failure, removing the Claude Code dependency, and dockerizing the template for platform-independent use.

1.1 User Requests This Session

1.1.1 1. Diagnose `lightgbm` build failure when running `bootstrap.py`

Error:

```
ninja: error: '/opt/homebrew/opt/libomp/lib/libomp.dylib', needed by 'lib_lightgbm.so',
missing and no known rule to make it
```

Root cause: System Python is 3.14. LightGBM has no pre-built wheel for Python 3.14 yet — pip falls back to compiling from source, which requires `libomp` (OpenMP). `libomp` was not installed on this Mac.

Immediate fix: `brew install libomp`

Longer-term fix: Dockerize with Python 3.11, which has pre-built wheels — no compilation, no `libomp` needed.

1.1.2 2. Understand `venv` / Docker relationship

User asked: is a `venv` created? Should it be? Should things be Dockerized?

Clarification provided: - The bootstrap script *did* create a `venv` (✓ Virtual environment created) but failed during pip install inside it - The `venv` lives at `.ml_staging_Temp-Insurance_<timestamp>/.venv/` - Docker is the better long-term approach — it replaces the need for host-level `venv` management, handles `libomp` automatically, and is platform-independent

1.1.3 3. Analyse builds_bootstrap repo for dockerization

Fetches and analysed: - bootstrap.py (201KB) — interactive setup wizard, ends with `claude .` - auto_pipeline.py (2,358 lines) — **fully standalone pure-Python ML pipeline, zero Claude dependency** - start.sh — bash wizard that also ends with `claude .` - Dockerfile.bootstrap — existed but was minimal (no system deps, no boosters) - requirements.txt — core ML stack only, no lightgbm/xgboost/catboost

Key finding: auto_pipeline.py already contained the entire ML pipeline (EDA → preprocessing → training → API generation → Docker → deploy) with no Claude calls anywhere. The docstring literally says "no Claude/AI required".

1.1.4 4. Decision: remove Claude dependency entirely

User confirmed goal: pipeline should run without `claude .`, platform-independent.

Decision: Replace `claude .` in start.sh and bootstrap.py with `python3 auto_pipeline.py`.

This is a two-line change in start.sh that removes the only Claude dependency.

1.1.5 5. Add LightGBM, XGBoost, CatBoost as standard libraries

User requested all three boosters be included even though they are large, because they are useful in ML.

Added to requirements.txt:

```
lightgbm>=4.3.0
xgboost>=2.0.0
catboost>=1.2.0
```

auto_pipeline.py already handles missing boosters gracefully (skips with a warning) so host installs without them still work.

1.1.6 6. Implement all changes in builds_bootstrap repo

Files changed:

File	Change
start.sh	Removed Node/Claude checks; added libomp macOS auto-install; Docker-aware paths via ML_OUTPUT_DIR env var; --system-site-packages venv in Docker; replaced <code>claude .</code> with <code>python3 auto_pipeline.py</code> ; simplified entry menu (removed Claude option)
requirements.txt	Added lightgbm, xgboost, catboost
Dockerfile.bootstrap	Full rewrite — Python 3.11-slim, libgomp1, GitHub CLI, all boosters pre-installed, ENV ML_OUTPUT_DIR=/output, entrypoint → bash start.sh
run.sh	New — one-command launcher: <code>docker build + docker run</code> with volume and GitHub auth mounts
.dockerignore	New — excludes .git, .venv, CSVs, model artifacts
README.md	Full rewrite — Claude-free, Docker-first workflow

Commits pushed: - 6e3e184 — main code changes - 862798f — docs updates (changelog.md, how_to_run.md)

1.1.7 7. Preserve Claude-based version as git tag

Before pushing, tagged the original HEAD:

```
git tag v1.0-claude
git push origin v1.0-claude
```

The Claude-based version is permanently accessible at: https://github.com/ramleo/builds_bootstrap/tree/v1.0-claude

Tags are found under the **Tags** tab in the branch dropdown on GitHub (not with branches).

Decision: keep existing repo (ramleo/builds_bootstrap) — no new repo needed. These are improvements to the same tool; splitting would fragment history and issues.

1.1.8 8. Explain ./run.sh vs ./start.sh

	./run.sh	./start.sh
Runs inside	Docker container	Your Mac
Python version	3.11 (pinned)	System Python (was 3.14 — caused errors)
lightgbm/xgboost/catboost	Pre-installed in image	Installed fresh (needs libomp on macOS)
Works on Windows	Yes (Docker Desktop)	No
First run	~5 min (image build, cached after)	Faster
Isolated from Mac	Fully	No

Recommendation: Use ./run.sh going forward.

1.1.9 9. Update all documentation

In builds_bootstrap repo (pushed to GitHub): - docs/changelog.md — v2.0.0 entry with full architecture change notes - docs/how_to_run.md — rewritten for Docker-first workflow, Claude references removed

Locally in Projects/docs/: - session_2026-05-31_docker-refactor.md — full session notes - ML_Portfolio_Guide.md — updated to v1.2: - Architecture diagram updated (removed Claude, added auto_pipeline.py) - Quickstart updated (Docker as Method A) - Model candidates table updated (added LightGBM, XGBoost, CatBoost) - Troubleshooting updated (removed Claude error row, added libomp row) - Changelog table updated (v2.0.0 row added) - ML_Portfolio_Guide.pdf — regenerated (113KB)

1.1.10 10. Add friendly Docker-not-running error to run.sh

User hit this raw error when running ./run.sh with Docker Desktop closed:

```
ERROR: Cannot connect to the Docker daemon at unix:///Users/wrks/.docker/run/docker.sock.
Is the docker daemon running?
```

Fix: Added a pre-flight docker info check to run.sh. If the daemon is unreachable, the script exits immediately with a clear message and step-by-step fix instead of the raw socket error.

Output shown to user:

```
ERROR: Docker is not running.
```

Fix:

1. Open Docker Desktop from your Applications folder
(or: open `-a Docker`)
2. Wait for the whale icon in the menu bar to stop animating (~30–60s)
3. Re-run: `./run.sh`

Files changed:

File	Change
run.sh	Added docker info pre-flight check with friendly error message
docs/changelog.md	v2.0.1 entry
docs/how_to_run.md	New troubleshooting row for Docker daemon error
Projects/docs/ML_Portfolio_Guide.md	v1.3 — new troubleshooting row, v2.0.1 changelog entry
Projects/docs/session_2026-05-31_docker-refactor.md	Patch noted in files-changed table

Commit pushed: 7ead56d — v2.0.1 on main

1.2 Final Architecture After This Session

User runs one command

↓
`./run.sh` (Docker) or `./start.sh` (host)

↓
 5 questions: project name, CSV path, target column, platform, GitHub

↓
 auto_pipeline.py runs automatically
 → EDA → Preprocessing → Train (sklearn + LightGBM + XGBoost + CatBoost)
 → GridSearchCV → best model → Evaluate → Save .pkl
 → FastAPI app + themed index.html + Dockerfile
 → Push to GitHub → Deploy

↓
 Done — no Claude, no AI subscription required

1.3 All Commits This Session

Repo	Commit	Description
builds_bootstrap	6e3e184	Remove Claude dependency — pipeline runs via auto_pipeline.py
builds_bootstrap	862798f	Update docs for v2.0.0 — changelog and how_to_run
builds_bootstrap	7ead56d	v2.0.1 — friendly Docker-not-running error in run.sh

Tag created: v1.0-`claude` ☐ original Claude-based version